Senior Java Engineer

Technical Vetting Assignment

Instructions & Requirements

Take Home Assignment: XML-to-JSON Content Transformation Service

Context

LexisNexis ingests high volumes of XML legal documents, validates them against schemas, transforms them into normalized structures, and publishes them for downstream search and AI/RAG pipelines. The French Content Systems team is modernizing these pipelines in the cloud and needs a senior IC who can design, own, and evolve Java/Spring services handling these transformations at scale.

You will build a small, production-style service that ingests XML, validates it, transforms it with XSLT, and publishes normalized artifacts suitable for downstream search/RAG. Focus on correctness, modularity, and an architecture that can scale and be operated in a cloud environment.

Requirements

Core technologies to use:

- Java 17+
- Spring Boot 3.x
- Saxon-HE (XSLT 3.0)
- Any one cloud environment assumptions (AWS, Azure, or GCP) in your design notes

You will define the API, module boundaries, error model, and data storage approach. Use the example data formats below to guide your implementation.

Task 1: Ingest ’ Validate ’ Transform ’ Publish

Build a Spring Boot service that:

- Accepts legal XML documents for processing. You choose how requests are submitted (single document vs. batches).
- Validates each XML document against the provided XSD (below). Invalid items should be recorded with diagnostics and not published as normalized output.
- Transforms valid XML to a normalized JSON record using XSLT executed by Saxon-HE. Also produce plain text suitable for AI/RAG (e.g., concatenated paragraphs or another approach you prefer).
- Publishes artifacts locally (e.g., filesystem) or in-memory for demo purposes. Key artifacts by a stable identifier to avoid duplicate outputs on repeated submissions of the same content.
- Provides a way to retrieve the status and outputs for processed items.

Example XML (namespaced):

```
<code class="language-xml"><?xml version="1.0" encoding="UTF-8"?>
<judgment xmlns="urn:lex:content:1">
  <header>
    <content_id>FR-2024-CA-000123</content_id>
    <title>Cour d'appel de Paris, 12 mars 2024, n° 20/01234</title>
    <court>Cour d'appel de Paris</court>
    <jurisdiction>FR</jurisdiction>
    <decision_date>2024-03-12</decision_date>
    <citations>
      <citation type="ECLI">ECLI:FR:CA12345</citation>
      <citation type="NOR">NOR:ABCD1234567</citation>
```

```
    </citations>
    <parties>
      <party role="appellant">Société ABC</party>
      <party role="respondent">M. Dupont</party>
    </parties>
  </header>
  <body>
    <section type="facts">
      <p id="p1">Le litige porte sur...</p>
    </section>
    <section type="reasons">
      <p id="p2">Considérant que...</p>
      <p id="p3">Attendu que...</p>
    </section>
    <section type="disposition">
      <p id="p4">Par ces motifs...</p>
    </section>
  </body>
</judgment>
```

Minimal XSD (you may extend as needed):

```
<code class="language-xml"><?xml version="1.0" encoding="UTF-8"?>
<xs:schema
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="urn:lex:content:1"
  xmlns="urn:lex:content:1"
  elementFormDefault="qualified">

  <xs:element name="judgment">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="header">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="content_id" type="xs:string"/>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="court" type="xs:string"/>
              <xs:element name="jurisdiction" type="xs:string"/>
              <xs:element name="decision_date" type="xs:date"/>
              <xs:element name="citations" minOccurs="0">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="citation" minOccurs="0" maxOccurs="unbounded">
                      <xs:complexType mixed="true">
                        <xs:simpleContent>
                          <xs:extension base="xs:string">
                            <xs:attribute name="type" type="xs:string" use="re-
quired"/>
                          </xs:extension>
                        </xs:simpleContent>
                      </xs:complexType>
                    </xs:element>
                  </xs:sequence>
```

```
                </xs:complexType>
              </xs:element>
              <xs:element name="parties" minOccurs="0">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="party" minOccurs="0" maxOccurs="unbounded">
                      <xs:complexType mixed="true">
                        <xs:simpleContent>
                          <xs:extension base="xs:string">
                            <xs:attribute name="role" type="xs:string" use="re-
quired"/>
                          </xs:extension>
                        </xs:simpleContent>
                      </xs:complexType>
                    </xs:element>
                  </xs:sequence>
                </xs:complexType>
              </xs:element>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="body">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="section" maxOccurs="unbounded">
                <xs:complexType>
                  <xs:sequence>
                    <xs:element name="p" maxOccurs="unbounded">
                      <xs:complexType mixed="true">
                        <xs:simpleContent>
                          <xs:extension base="xs:string">
                            <xs:attribute name="id" type="xs:ID" use="required"/>
                          </xs:extension>
                        </xs:simpleContent>
                      </xs:complexType>
                    </xs:element>
                  </xs:sequence>
                  <xs:attribute name="type" type="xs:string" use="required"/>
                </xs:complexType>
              </xs:element>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Normalized JSON target (example shape; adapt as needed):

```
<code class="language-json">{
  "content_id": "FR-2024-CA-000123",
  "title": "Cour d'appel de Paris, 12 mars 2024, n° 20/01234",
  "court": "Cour d'appel de Paris",
```

```
  "jurisdiction": "FR",
  "decision_date": "2024-03-12",
  "citations": [
    {"type": "ECLI", "value": "ECLI:FR:CA12345"},
    {"type": "NOR", "value": "NOR:ABCD1234567"}
  ],
  "parties": [
    {"role": "appellant", "name": "Société ABC"},
    {"role": "respondent", "name": "M. Dupont"}
  ],
  "paragraphs": [
    {"id": "p1", "section": "facts", "text": "Le litige porte sur..."},
    {"id": "p2", "section": "reasons", "text": "Considérant que..."},
    {"id": "p3", "section": "reasons", "text": "Attendu que..."},
    {"id": "p4", "section": "disposition", "text": "Par ces motifs..."}
  ],
  "full_text": "Le litige porte sur... Considérant que... Attendu que... Par ces mo-
tifs..."
}
```

Notes:

- Use XSLT (via Saxon-HE) to produce the normalized JSON shown above (or your close variant).
- You decide how to store artifacts and how clients discover them later.
- Expect repeated submissions of the same `content_id`. Handle this gracefully so that only one published artifact represents a given `content_id`.

Task 2: Batch processing, scale signals, and operability

Add capabilities to run the transformation over a batch of XML files and demonstrate the service can handle a modest workload on a laptop:

- Provide a simple way to submit a folder of XML files (e.g., a batch endpoint or a CLI wrapper that calls your service).
- Process multiple documents concurrently. Make the level of concurrency configurable without code changes.
- Expose health/readiness information and basic runtime metrics useful to operators (e.g., counts, durations, or similar).
- Avoid unnecessary memory growth when processing larger inputs. Favor an approach that works well when documents get big.

You decide how to model work tracking, how to expose metrics, what to log, and how to guard against noisy or malformed input.

Task 3: Cloud-ready packaging and evolution plan

Prepare the service for a straightforward deployment to one cloud of your choice:

- Containerize the service.
- Externalize configuration (e.g., paths, concurrency, output location) via environment-based settings.
- In SOLUTION.md, outline how you would run this in your chosen cloud, including:
    - Where you would store inputs and outputs
    - How you would trigger processing at volume
    - How you would monitor the service
    - How you would prevent duplicate publishing when the same `content_id` is submitted multiple times
    - How you would evolve this design to feed a RAG pipeline later (what additional artifacts or metadata you would produce)

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A README.md with instructions to run locally.

- A SOLUTION.md describing the design and trade-offs you made.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Highlight interesting technical implementations

Time Estimate

This should take ±4 hours to complete, but you can take as long as you need.